

1) Pseudocode: Simplified TLS Handshake

BEGIN

// Client initiates connection

CLIENT:

Generate random ClientRandom

Send "ClientHello"

- Supported TLS versions**
- Supported cipher suites**
- ClientRandom**

// Server responds

SERVER:

Generate random ServerRandom

Select TLS version and cipher suite

Send "ServerHello"

- Selected TLS version**
- Selected cipher suite**
- ServerRandom**

// Server authentication

SERVER:

Send DigitalCertificate

- Server public key**
- Server identity**
- CA signature**

CLIENT:

Receive DigitalCertificate

IF Certificate is valid AND

Certificate is not expired AND

Certificate is issued by a trusted CA AND

Server identity matches certificate

THEN

Accept server authentication

ELSE

Terminate connection

END IF

// Session key generation

CLIENT:

Generate PreMasterSecret

Encrypt PreMasterSecret using

ServerPublicKey

Send encrypted PreMasterSecret to SERVER

SERVER:

Decrypt PreMasterSecret using ServerPrivateKey

// Both sides independently derive the session key

CLIENT:

SessionKey = DeriveKey(

PreMasterSecret,

ClientRandom,

ServerRandom

)

SERVER:

SessionKey = DeriveKey(

PreMasterSecret,

ClientRandom,

ServerRandom

)

// Verify handshake

CLIENT:

Generate FinishedMessage

Encrypt FinishedMessage using SessionKey

Send FinishedMessage

SERVER:

Decrypt FinishedMessage using SessionKey

Verify handshake parameters

SERVER:

Generate FinishedMessage

Encrypt FinishedMessage using SessionKey

Send FinishedMessage

CLIENT:

Decrypt and verify FinishedMessage

// Secure data exchange

CLIENT:

Data = "Confidential Message"

EncryptedData = Encrypt(Data, SessionKey)

Send EncryptedData

SERVER:

Receive EncryptedData

Data = Decrypt(EncryptedData, SessionKey)

IF integrity check is successful

Accept Data

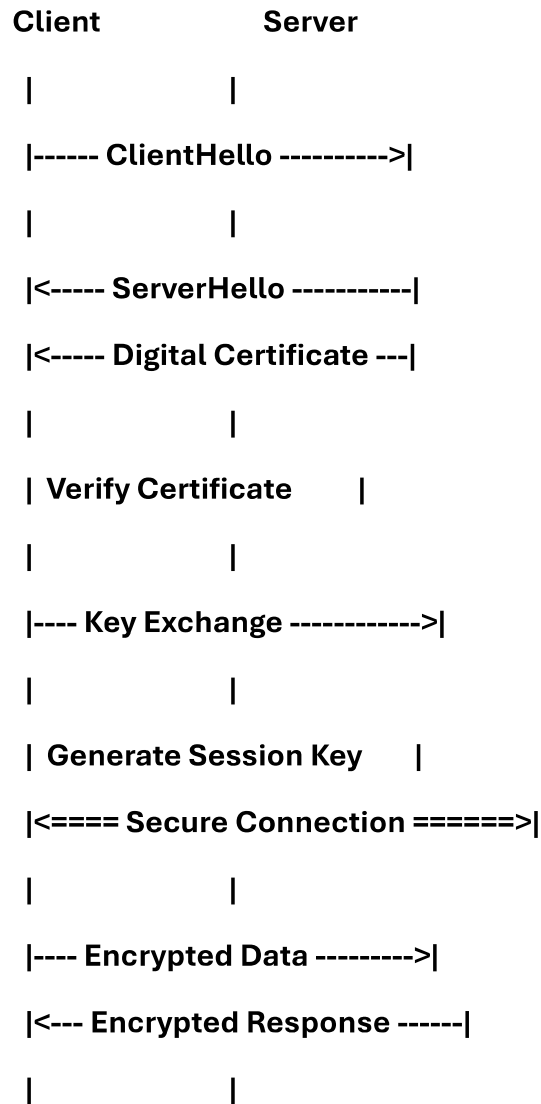
ELSE

Reject Data

END IF

END

Simplified Flow



Note: This is a simplified educational model. Modern TLS 1.3 normally uses an ephemeral Diffie–Hellman key exchange (such as ECDHE) rather than sending an encrypted premaster secret with the server's RSA public key. The certificate primarily authenticates the server; the ephemeral key exchange establishes the shared session secret.

2) Pseudocode: Firewall Rule Engine

BEGIN

// Define firewall rule structure

RULE:

sourceIP

destinationIP

port

protocol

action // ALLOW or BLOCK

// Create firewall rules

RULES = [

(ANY, "192.168.1.10", 80, "TCP", ALLOW),

(ANY, "192.168.1.10", 443, "TCP", ALLOW),

("10.0.0.5", ANY, 22, "TCP", BLOCK),

(ANY, ANY, ANY, "ICMP", BLOCK)

]

// Function to check whether a packet matches a rule

FUNCTION MATCH(packet, rule):

IF rule.sourceIP != ANY AND

packet.sourceIP != rule.sourceIP

THEN

RETURN FALSE

END IF

IF rule.destinationIP != ANY AND

packet.destinationIP != rule.destinationIP

THEN

RETURN FALSE

END IF

```
IF rule.port != ANY AND
    packet.destinationPort != rule.port
THEN
    RETURN FALSE
END IF

IF rule.protocol != ANY AND
    packet.protocol != rule.protocol
THEN
    RETURN FALSE
END IF

RETURN TRUE
END FUNCTION

// Function to process incoming packet
FUNCTION PROCESS_PACKET(packet):
    PRINT "Incoming Packet:"
    PRINT "Source IP:", packet.sourceIP
    PRINT "Destination IP:", packet.destinationIP
    PRINT "Port:", packet.destinationPort
    PRINT "Protocol:", packet.protocol

    // Check rules in order
    FOR each rule IN RULES:
        IF MATCH(packet, rule) == TRUE THEN
            IF rule.action == ALLOW THEN
                PRINT "Action: ALLOW"
                RETURN ALLOW
            ELSE IF rule.action == BLOCK THEN
                PRINT "Action: BLOCK"
                RETURN BLOCK
```

```
        END IF
    END IF
END FOR

// Default security policy
PRINT "No matching rule"
PRINT "Action: BLOCK"
RETURN BLOCK
END FUNCTION

// Main program
WHILE TRUE:

    packet = RECEIVE_PACKET()
    result = PROCESS_PACKET(packet)

    IF result == ALLOW THEN
        FORWARD_PACKET(packet)
    ELSE
        DROP_PACKET(packet)
    END IF
END WHILE
END
```

Example

Suppose an incoming packet has:

Source IP : 10.0.0.5

Destination IP : 192.168.1.10

Port : 80

Protocol : TCP

The firewall checks the rules from top to bottom. If the packet matches an ALLOW rule, it is forwarded. If it matches a BLOCK rule, it is dropped.

The important security principle is the default-deny rule: if no rule matches the packet, the firewall blocks it. This prevents unknown or unauthorized traffic from entering the network.

3) Pseudocode: Malware Behavior Detection and Classification

BEGIN

FUNCTION CLASSIFY_MALWARE(activity):

// Collect behavioral information

propagation = activity.propagation_method

file_changes = activity.file_modification_pattern

user_action = activity.requires_user_interaction

// Check for Worm behavior

IF propagation == "AUTOMATIC_NETWORK_PROPAGATION" AND

user_action == FALSE THEN

RETURN "WORM"

// Check for Virus behavior

ELSE IF file_changes == "ATTACHES_TO_OR_MODIFIES_FILES" AND

propagation == "FILE_BASED" THEN

RETURN "VIRUS"

// Check for Trojan behavior

ELSE IF user_action == TRUE AND

propagation == "DISGUISED_AS_LEGITIMATE_PROGRAM" THEN

RETURN "TROJAN"

// Additional behavioral analysis

ELSE

score_virus = 0

score_worm = 0

score_trojan = 0

IF file_changes == "MODIFIES_EXECUTABLE_FILES" THEN

score_virus = score_virus + 2

END IF

```
IF propagation == "SPREADS_TO_OTHER_HOSTS_AUTOMATICALLY" THEN
    score_worm = score_worm + 3
END IF

IF user_action == TRUE THEN
    score_trojan = score_trojan + 2
END IF

IF activity.disguised_as_legitimate_software == TRUE THEN
    score_trojan = score_trojan + 3
END IF

IF activity.scans_network_for_targets == TRUE THEN
    score_worm = score_worm + 2
END IF

IF activity.self_replicates_inside_files == TRUE THEN
    score_virus = score_virus + 3
END IF

// Select highest behavioral score

IF score_worm > score_virus AND
    score_worm > score_trojan THEN
    RETURN "WORM"
ELSE IF score_virus > score_worm AND
    score_virus > score_trojan THEN
    RETURN "VIRUS"
ELSE IF score_trojan > score_worm AND
    score_trojan > score_virus THEN
    RETURN "TROJAN"
ELSE
    RETURN "UNKNOWN / REQUIRES FURTHER ANALYSIS"
```

```
    END IF

END IF

END FUNCTION

// Main program

activity = MONITOR_SYSTEM()

classification = CLASSIFY_MALWARE(activity)

PRINT "Malware Classification:", classification

// Recommended response

IF classification == "WORM" THEN

    ISOLATE_INFECTED_SYSTEM()

    BLOCK_SUSPICIOUS_NETWORK_CONNECTIONS()

    SCAN_OTHER_NETWORK_HOSTS()

ELSE IF classification == "VIRUS" THEN

    QUARANTINE_INFECTED_FILES()

    SCAN_EXECUTABLE_FILES()

    RESTORE_CLEAN_FILES()

ELSE IF classification == "TROJAN" THEN

    TERMINATE_SUSPICIOUS_PROCESS()

    QUARANTINE_MALICIOUS_PROGRAM()

    INVESTIGATE_USER_EXECUTION_SOURCE()

ELSE

    GENERATE_SECURITY_ALERT()

    SEND_TO_SECURITY_ANALYST()

END IF

END
```

Classification Logic

Behavior	Virus	Worm	Trojan
Modifies/attaches to files	✓	✗	Usually ✗
Automatically spreads through network	✗	✓	✗
Requires user interaction	Usually	✗	Often ✓
Disguises itself as legitimate software	✗	✗	✓
Scans network for other systems	✗	✓	Usually ✗
Self-replicates	✓	✓	✗

Decision:

- File modification + attachment → Virus
- Automatic network propagation without user interaction → Worm
- Disguised software + user execution → Trojan
- Unclear behavior → Further analysis required

4) Pseudocode: Hybrid IDS Using Signature and Anomaly Detection

BEGIN

// Load known attack signatures

SIGNATURE_DATABASE = LOAD_SIGNATURES()

// Build normal traffic profile

NORMAL_PROFILE = LOAD_NORMAL_TRAFFIC_PROFILE()

// Set detection thresholds

SIGNATURE_THRESHOLD = 1

ANOMALY_THRESHOLD = 0.80

FUNCTION SIGNATURE_DETECTION(packet):

FOR each signature IN SIGNATURE_DATABASE:

IF MATCH(packet, signature) == TRUE THEN

RETURN TRUE

END IF

END FOR

RETURN FALSE

END FUNCTION

FUNCTION ANOMALY_DETECTION(packet):

// Extract traffic features

features = EXTRACT_FEATURES(packet)

// Calculate deviation from normal behavior

anomaly_score = CALCULATE_ANOMALY_SCORE(

features,

NORMAL_PROFILE

)

IF anomaly_score >= ANOMALY_THRESHOLD THEN

RETURN TRUE, anomaly_score

ELSE

```
        RETURN FALSE, anomaly_score
    END IF
END FUNCTION

FUNCTION PROCESS_PACKET(packet):
    // Signature-based detection
    signature_result = SIGNATURE_DETECTION(packet)

    // Anomaly-based detection
    anomaly_result, anomaly_score =
        ANOMALY_DETECTION(packet)

    // Combine detection results
    IF signature_result == TRUE AND
        anomaly_result == TRUE THEN
        severity = "CRITICAL"
        reason = "Known attack + abnormal behavior"
    ELSE IF signature_result == TRUE THEN
        severity = "HIGH"
        reason = "Known attack signature detected"
    ELSE IF anomaly_result == TRUE THEN
        severity = "MEDIUM"
        reason = "Abnormal network behavior detected"
    ELSE
        severity = "NORMAL"
        reason = "No suspicious activity"
    END IF

    // Generate IDS response
    IF severity != "NORMAL" THEN
        CREATE_ALERT(
            packet,
```

```

        severity,
        reason,
        anomaly_score
    )
    LOG_EVENT(packet, severity, reason)
END IF
END FUNCTION

// Main IDS monitoring loop
WHILE NETWORK_IS_ACTIVE():
    packet = CAPTURE_PACKET()
    PROCESS_PACKET(packet)
END WHILE
END

```

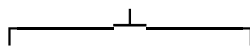
Detection Flow

Incoming Network Packet

↓

Extract Features

↓



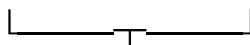
↓ ↓

Signature Anomaly

Detection Detection

↓ ↓

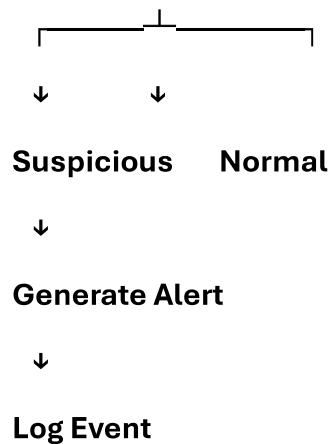
Known Attack? Abnormal?



↓

Correlate Results

↓



How It Works

1. Signature-based detection:

The IDS compares network traffic against a database of known attack signatures. It is highly effective for detecting known attacks, but may miss new attacks.

2. Anomaly-based detection:

The IDS compares current traffic against a normal behavior profile. Significant deviations are flagged as suspicious, allowing detection of potentially unknown attacks.

3. Hybrid decision:

Using both methods improves coverage:

- Signature + anomaly detected → Critical
- Signature only → High
- Anomaly only → Medium
- Neither detected → Normal

This hybrid approach combines the accuracy of signature detection for known threats with the adaptability of anomaly detection for previously unseen or evolving attacks.

5) Pseudocode: Intrusion Prevention System (IPS)

BEGIN

// Load security information

SIGNATURE_DATABASE = LOAD_ATTACK_SIGNATURES()

THREAT_INTELLIGENCE = LOAD_MALICIOUS_IP_LIST()

FUNCTION INSPECT_TRAFFIC(packet):

// Extract packet information

sourceIP = packet.sourceIP

destinationIP = packet.destinationIP

port = packet.destinationPort

protocol = packet.protocol

payload = packet.payload

// Check whether source is known to be malicious

IF sourceIP IN THREAT_INTELLIGENCE THEN

RETURN "MALICIOUS", "Known malicious source"

END IF

// Compare packet against known attack signatures

FOR each signature IN SIGNATURE_DATABASE:

IF MATCH(payload, signature) == TRUE THEN

RETURN "MALICIOUS", "Attack signature detected"

END IF

END FOR

// Detect suspicious behavior

IF IS_ABNORMAL(packet) == TRUE THEN

RETURN "SUSPICIOUS", "Abnormal network behavior"

END IF

RETURN "NORMAL", "No threat detected"

END FUNCTION

FUNCTION LOG_INTRUSION(packet, reason):

```
    logEntry = CREATE_LOG(  
        timestamp,  
        packet.sourceIP,  
        packet.destinationIP,  
        packet.destinationPort,  
        packet.protocol,  
        reason  
    )
```

```
    SAVE_LOG(logEntry)
```

END FUNCTION

FUNCTION BLOCK_TRAFFIC(packet):

```
    DROP_PACKET(packet)  
  
    // Optionally block the source temporarily  
  
    ADD_TO_BLOCKLIST(packet.sourceIP)
```

END FUNCTION

// Main IPS monitoring loop

WHILE NETWORK_IS_ACTIVE():

```
    packet = CAPTURE_PACKET()  
  
    status, reason = INSPECT_TRAFFIC(packet)  
  
    IF status == "MALICIOUS" THEN  
  
        BLOCK_TRAFFIC(packet)  
  
        LOG_INTRUSION(packet, reason)  
  
        PRINT "Malicious request blocked"  
  
    ELSE IF status == "SUSPICIOUS" THEN  
  
        LOG_INTRUSION(packet, reason)
```

```

// Monitor or block depending on security policy
IF HIGH_CONFIDENCE_THREAT(packet) == TRUE THEN
    BLOCK_TRAFFIC(packet)
    PRINT "Suspicious request blocked"
ELSE
    FORWARD_PACKET(packet)
    PRINT "Suspicious request monitored"
END IF
ELSE
    FORWARD_PACKET(packet)
END IF
END WHILE
END

```

Working Flow

Incoming Packet

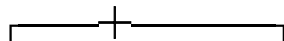
↓

Capture Traffic

↓

Inspect Packet

↓



↓ ↓ ↓

Known Suspicious Normal

Attack Behavior Traffic

↓ ↓ ↓

Block Analyze Allow

↓ ↓

Log Block if

High Risk

Key Functions

- **Real-time monitoring:** Continuously captures incoming network traffic.
- **Signature detection:** Compares traffic against known attack signatures.
- **Threat intelligence:** Checks source IPs against known malicious sources.
- **Behavior analysis:** Identifies unusual or suspicious traffic.
- **Automatic blocking:** Drops confirmed malicious packets before they reach internal systems.
- **Logging:** Records the source, destination, protocol, time, and reason for every blocked intrusion.
- **Default handling:** Legitimate traffic is forwarded normally.

This design allows the IPS to detect → decide → block → log malicious traffic automatically while minimizing unnecessary disruption to legitimate network communication.